



Universidad de Jaén

Tema 2.1 Desarrollo web en Jakarta EE

Arquitectura, Servlets y JSP

José Ramón Balsas Almagro
Departamento de Informática
Universidad de Jaén
jrbalsas@ujaen.es

<http://tiny.cc/dawuja>

Este obra está bajo una Licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional.

v 1.7

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Objetivos

- Conocer la **Arquitectura JakartaEE** y conceptos relacionados
- Conocer el **concepto de Servlet**, sus funciones y su relación con la programación en el lado del servidor web
- Conocer la **organización de ficheros** de una aplicación web
- Saber **procesar una transacción HTTP** desde un Servlet
- Conocer el concepto de **página JSP**, sus funciones y su relación con los servlets



Arquitectura *Java Enterprise Edition*, Java EE, Jakarta EE

- Extensión de la plataforma *Java Standard Edition* (Java SE) para desarrollo de Aplicaciones Empresariales

• Elementos

- Entorno de ejecución o contenedores
- Conjunto de módulos o bibliotecas orientados a servicios



• Especificación

- **Objetivos:** estandarización, estabilidad y compatibilidad
- Establecida mediante *Eclipse Specification Community Process, ESCP* (antiguo JCP)
- Define el API de cada módulo y sus características mediante una especificación numérica (*antiguo Java Specification Requests, JSR*, hasta JEE8)
- Versión estable: JakartaEE 10 <https://jakarta.ee> (septiembre 2022)

• Implementaciones compatibles

- Deben superar los tests *Technology Compatibility Kit, TCK*
- Servidor de aplicaciones Glassfish 7.0 (referencia Jakarta10), Glassfish 6 (Jakarta9)
- Otras: Payara, Wildfly, OpenLiberty, Oracle Weblogic, IBM Websphere,...

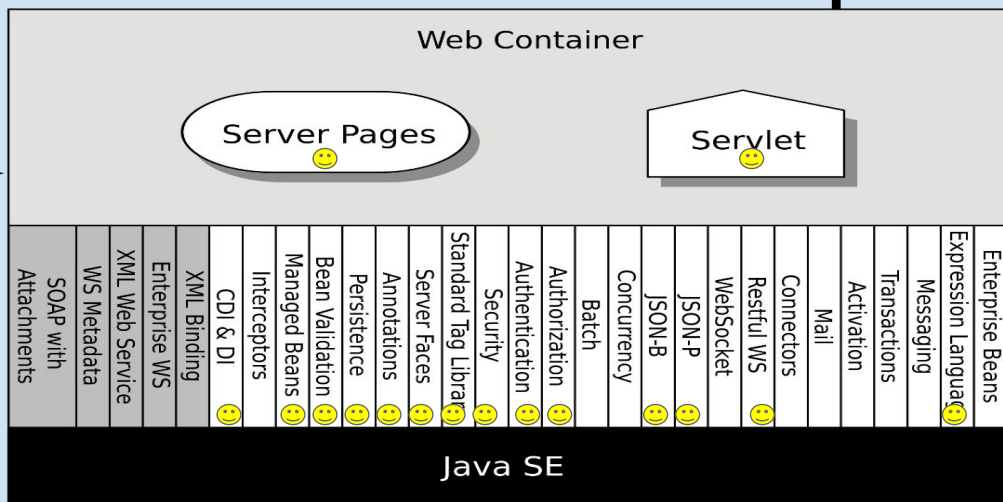


3

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Módulos Jakarta EE 10 - Web profile



Fuente: Eclipse (2022) Jakarta EE 10 [Platform Overview](#)

4

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Payara Server



<https://www.payara.fish/>

- Derivado de GlassFish. OpenSource
- Compatible JakartaEE 10 y OpenJDK LTS 11, 17 y 21
- Compatible con Eclipse Microprofile
 - Soporte para arquitecturas en la nube
 - Desarrollo de microservicios basados en JEE
- Modos de ejecución
 - Despliegue en servidor aplicaciones independiente (aplicaciones WAR)
`application.war`
 - Servidor embebido en JAR de aplicación (uberJar)
 - `$ java -jar application.jar`
- Comunidad de desarrollo
- Actualizaciones periódicas de versiones
- Modelo de negocio basado en soporte técnico

5

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Apache Tomcat



- Servidor web y contenedor de *Servlets*
- *Proyecto Open Source* desarrollado bajo el apoyo de la *Apache Software Foundation*
- Especificaciones JakartaEE 10 soportadas Tomcat 10
 - Servlet 6.0
 - JSP 3.1
 - EL 5.0
 - WebSocket 2.1
 - Otros módulos JEE deben añadirse expresamente
- Usos
 - Aplicaciones web con requisitos técnicos bajos
 - Uso en frameworks especializados, e.g. Spring-Boot

6

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Servlets

• Java Servlets

- 1997, especificación 1.0 por *Sun Microsystems*
- 2001, especificación 2.3 (Java Specification Request, JSR 53)
- Septiembre, 2017, especificación 4.0 (JSR 369) → JEE8
- Mayo, 2022, especificación 6.0 → JakartaEE 10

• Definición

- Un **Servlet** es una clase que implementa la interfaz **jakarta.servlet.Servlet** o hereda de **jakarta.servlet.http.HttpServlet**
 - Atienden peticiones HTTP y generan respuestas HTTP
 - **Métodos** a redefinir según tipo de petición HTTP que se desea atender
 - `doGet` (`HttpServletRequest req`, `HttpServletResponse resp`)
 - `doPost` (`HttpServletRequest req`, `HttpServletResponse resp`)
 - `doPut` (`HttpServletRequest req`, `HttpServletResponse resp`)
 - `doDelete` (`HttpServletRequest req`, `HttpServletResponse resp`)
- Otros tipos de **objetos relacionados**
 - **Filters**, pueden manipular información http
 - **Event Listeners**, detectan eventos en el ciclo de vida de una aplicación web: inicio aplicación, inicio de sesión, ...

7

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Estructura básica de un Servlet

```
@WebServlet("/cliente") //Enrutado de peticiones a URL http://hostname/app/cliente
public class ClienteServlet extends HttpServlet {
```

```
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //recuperar datos de objeto request y generar salida en objeto response
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //recuperar datos de objeto request y generar salida en objeto response
    }

    @Override
    protected void doPut(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    }

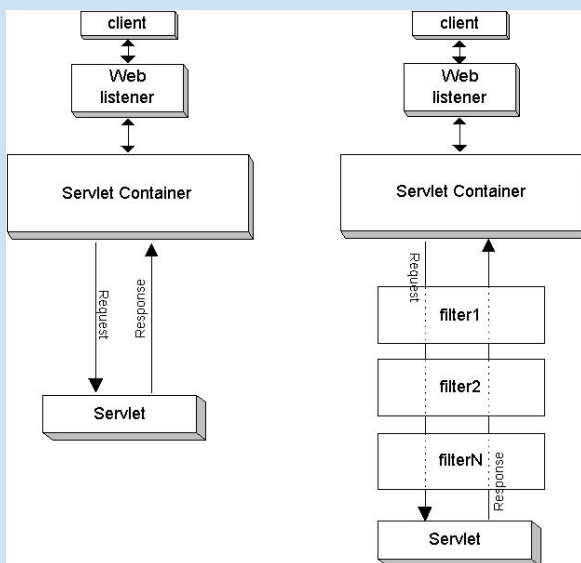
    @Override
    protected void doDelete(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    }
}
```

8 }

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Procesamiento petición HTTP



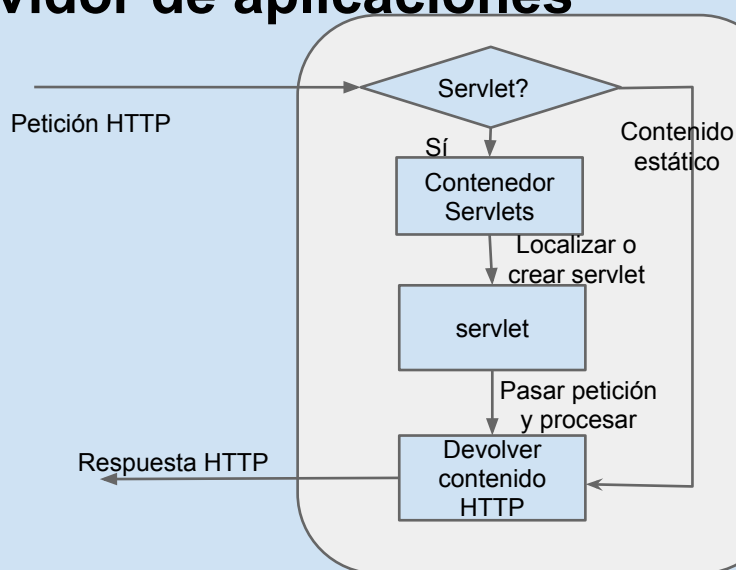
9

Fuente: Oracle Application Server Documentation Library

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Procesamiento petición HTTP en Servidor de aplicaciones



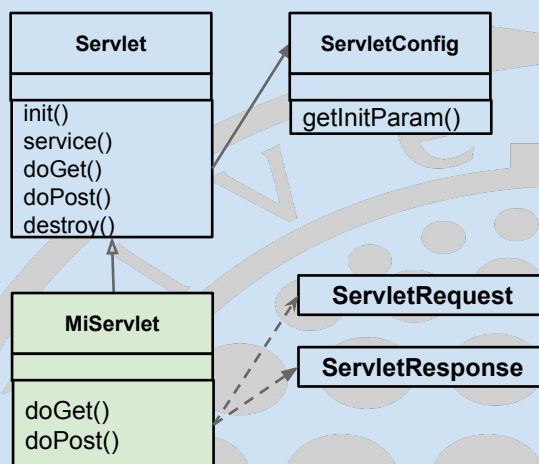
10

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ciclo de vida de un Servlet

- **init()**, se ejecuta al crear el servlet
- **service()**, se ejecuta para cada petición HTTP (hebra). Ejecuta un método específico según el tipo:
 - **doGet()**, **doPost()**, ...
 - Parámetros: **HttpServletRequest**, **HttpServletResponse** para gestionar la petición
- **destroy()**, se ejecuta al destruir el servlet. e.g. al parar el servidor



11

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Registro del Servlet

- Cada Servlet estará asociado en su contenedor a una o varias URLs (enrutado de urls)
- Formas de enrutar una URL a un Servlet
 - Configuración de la aplicación web

- Fichero WEB-INF/web.xml (método clásico)

```

<servlet>
  <servlet-name>MiServlet</servlet-name>
  <servlet-class>daw.MiServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>MiServlet</servlet-name>
  <url-pattern>/dawtest</url-pattern>
</servlet-mapping>

```

- Anotaciones en el servlet (especificación 3.0, **recomendada**)

```

@WebServlet("/dawtest")
class MiServlet extends HttpServlet

```

- Las direcciones son relativas a url de la app. web (*context path*):

- <http://host:8080/appName/dawtest>

12

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo de Servlet

```
@WebServlet("/dawtest")
public class MinServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        try {
            String doctxt = ""
                <!DOCTYPE html>
                <html><head><title>Servlet Mínimo</title></head><body>
                <h1>%s</h1>
                </html></body>
            ""
            .formatted( "Hola Mundo" );
            out.println(doctxt);
            // response.setStatus( 200 ); //Default response status OK (200)
        } finally {
            out.close();
        }
    }

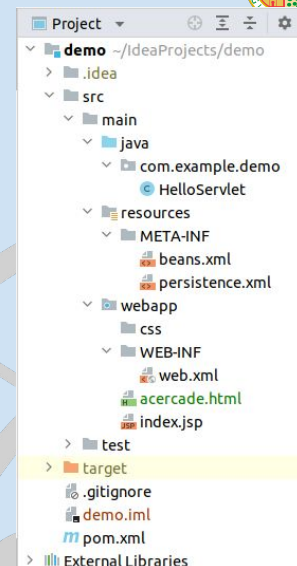
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        ...
    }
}
```

```
package daw;
```

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

Contenidos de una App. Web

- **Contenidos públicos, (Carpeta webapp ...)**
 - Estáticos: html, js, css, imágenes, ...
 - Plantillas: .jsp, .xhtml, .ftl, etc.
- **Contenidos privados**
 - Carpeta **WEB-INF** (no accesible vía web)
 - **web.xml**, descriptor de despliegue aplicación. Información sobre servlets, parámetros de inicialización, otros componentes, etc. (opcional esp. 3.0)
 - Otros contenidos privados, e.g layout, configuraciones adicionales para módulos
 - Carpeta **META-INF** (no accesible vía web)
 - e.g. persistence.xml, configuración de acceso a BD con JPA
 - **Otros contenidos:**
 - Fuentes de la aplicación, e.g. Servlets
 - Paquetes utilizados y dependencias, e.g. módulos JEE
 - Otros ficheros de configuración



El IDE gestiona los detalles de estructura de contenidos

Proceso de creación y despliegue de una app web

- **Empaquetado (package)**

- Una aplicación web JEE se empaqueta en un fichero .WAR (versión JAR para Web)
- Contiene clases compiladas (.class), dependencias, contenidos estáticos (webapp) y ficheros de configuración (WEB-INF, ...)

- **Despliegue (deploy)**

- El fichero .war se envía al servidor de aplicaciones, donde se descomprime
 - Durante el desarrollo no es necesario generar el war (*war exploded*)
- El servidor instancia los objetos de la aplicación
- Enruta las peticiones http a objetos, servlets, y contenidos estáticos

15

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Procesando la petición HTTP

- **Una única instancia de un Servlet** atiende todas las peticiones HTTP
 - El acceso a sus atributos deben estar sincronizado
- **Para cada petición HTTP** se crea una *hebra* del objeto Servlet y se llama a su método *service* con dos parámetros:
 - **HttpServletRequest**, datos de la petición HTTP
 - **HttpServletResponse**, generar respuesta HTTP
- *service* delega la petición a métodos **doGet**, **doPost**, ...
- **Funciones del objeto HttpServletRequest**
 - Recuperación de parámetros de petición:
 - de Formularios: e.g. `<input name="nombre">`
 - de Query-string: e.g. `http://host/app/servlet?id=25`
 - Recuperación de encabezados:
 - e.g. Accept: text/html, application/json, application/xml
 - Datos de sesión y cookies

16

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Métodos de Utilidad HttpServletRequest

Acceso a [@WebServlet\("/usuario/*"\)](http://www.ujaen.es/dawapp/usuario/visualiza?nombre=Alberto)

Tipo	Descripción
<code>getServerName()</code>	www.ujaen.es
<code>getServerPort()</code>	80
<code>getContextPath()</code>	/dawapp
<code>getServletPath()</code>	/usuario
<code>getMethod()</code>	GET
<code>getQueryString()</code>	nombre=alberto
<code>getRequestURL()</code>	http://www.ujaen.es/dawapp/usuario/
<code>getPathInfo()</code>	/visualiza
<code>getRemoteAddr()</code>	e.g. 150.210.155.37
<code>getRemoteHost()</code>	aula37.ujaen.es

17

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Métodos HttpServletRequest: recuperación de parámetros

- **Recordatorio: ¿Cómo se reciben los parámetros?**
 - GET: <http://.....?param1=value1¶m2=value2>
 - POST: [param1=value1¶m2=value2](#) en cuerpo de petición
- **`String getParameter("nombre")`**
 - Valor del parámetro (también "") o null si no existe
 - Solo primera ocurrencia si hay varios
- **`String[] getParameterValues("nombre")`**
 - Array con valores de parámetros con el mismo nombre. e.g. *Select* Múltiple o *checkboxButtons*
- **`Enumeration<String> getParameterNames()`**
 - Nombres de todos los parámetros
- **`Map<String,String[]> getParameterMap()`**
 - Almacena un vector de valores (string) por cada nombre de parámetro

18

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Recuperación de parámetros

```
@WebServlet("/param")
public class Test extends HttpServlet {

    protected void
        processRequest(HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {

        //Se ejecuta tanto para GET como POST
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        try {
            String doctxt;
            doctxt = "<!DOCTYPE html>"
                + "<html><head>"
                + "<title>Servlet Mínimo</title>"
                + "</head><body>"
                + Util.verParam(request)
                + "</html></body>";
            out.println(doctxt);
        } finally {
            out.close();
        }
    }
}
```

```
package daw.util;

import java.util.Enumeration;
import jakarta.servlet.http.HttpServletRequest;

public class Util {
    public static String verParam(HttpServletRequest request) {
        String txt;
        txt = "";
        Enumeration<String> pNames;
        pNames=request.getParameterNames();
        while (pNames.hasMoreElements()) {
            String pName=pNames.nextElement();
            String pValues[];
            pValues=request.getParameterValues(pName);
            for (String valor: pValues){
                txt+=String.format("<b>%s</b>:%s<br>",
                    pName,valor);
            }
        }
        return txt;
    }
}
```

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Tratamiento de parámetros

- Los **parámetros enviados** desde un formulario o en un query-string deben ser **tratados** antes de su procesamiento:
 - Pueden no existir (null)
 - e.g. checkbox no seleccionado
 - No tener ningún contenido (cadena vacía)
 - Tener espacios en blanco antes o después
 - e.g " Paco "
 - **Por seguridad, no aceptar ningún carácter no esperado**

• Ejemplo:

```
string valorParam (string param, string defecto) {
    if (param==null || param.trim().equals("");
        param=defecto;
    else param=param.trim();
    return param;
}
```

```
string usuario = valorParam(
    request.getParameter("nombre"),
    "Desconocido"
);
```

20

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Métodos HttpServletRequest: recuperación de encabezados HTTP

- Genéricos

- `string getHeader("nombre")`
 - Valor del encabezado o null si no existe
 - Solo primera ocurrencia si hay varios
- `Enumeration<String> getHeaders("nombre")`
 - Enumeración con valores de encabezados con el mismo nombre.
- `Enumeration<String> getHeaderNames()`
 - Nombres de todos los encabezados recibidos

Similares a los existentes para parámetros

- Específicos

- `int getContentLength()`
- `String getContentType()`
- `long getDateHeader("nombre")`
 - Para campos de tipo fecha realiza una conversión
- `int getIntHeader("nombre")`
 - Para campos de tipo int realiza una conversión

21

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Generando la respuesta HTTP

- Representada por objeto `HttpServletResponse`

- Funcionalidades

- Definir estado de la respuesta HTTP
 - e.g. 200 (Ok), 404 (Not Found)
- Crear redirecciones a otras URLs (e.g. después de POST)

- Métodos

- `setStatus(código_estado)`
 - Devuelve código de estado según HTTP
 - Constantes en `HttpServletResponse`:
 - `HttpServletResponse.SC_OK`, `SC_NOT_FOUND`, etc.
- `sendError(código,"Mensaje")`
 - Muestra una página de error con el mensaje
- `sendRedirect("nueva_url")`
 - Redirección del navegador a una nueva URL

22

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo de manejo de encabezados

```
@Override
protected void doGet(HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {
```

```
    response.setContentType("text/html;charset=UTF-8");
```

```
    float val1, val2, result=0;
    String error="";
```

```
    try {
        val1=Float.valueOf(request.getParameter("val1"));
        val2=Float.valueOf(request.getParameter("val2"));
        String operation=request.getParameter("operation");
```

```
        switch (operation) {
            case "add": result=val1+val2; break;
            case "min": result=val1-val2; break;
            case "mul": result=val1*val2; break;
            case "div": result=val1/val2; break;
            default: error="Invalid Operation";
```

```
        }
    } catch (NumberFormatException e) {
        error="Operators are not numbers";
```

```
    }
    http://192.170.2.100:8080/CalculatorService/calculator?val1=10&val2=20&operation=pow
```

```
    if (!error.isEmpty())
        response.sendError(HttpServletResponse.SC_BAD_REQUEST
            ,error); //Send 400 status code
```

```
    if ( !Float.isFinite(result) ) { //Rickroll user :-}
        response.sendRedirect("https://youtu.be/dQw4w9WgXcQ");
        return;
    }
    try (PrintWriter out = response.getWriter()) {
        out.println("<!DOCTYPE html><html><body>"
            +"Result:"+Float.toString(result)
            +"<br/>Access from: "+request.getRemoteAddr()
            +"<br/>Browser: "+request.getHeader("User-Agent")
            +"</body></html>");
    }
```

Estado HTTP 400 - Invalid Operation

type Informe de estado

mensaje Invalid Operation

descripción El requerimiento enviado por el cliente era sintácticamente incorrecto.

Apache Tomcat/8.5.11

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Métodos HttpServletResponse: creación de encabezados HTTP

● Genéricos

- **set/addHeader("nombre", "valor")**
 - Establece o reemplaza/Añade valor de encabezado
- **set/addDateHeader("nombre", timestamp)**
 - Establecer/añade encabezados de tipo fecha
- **set/addIntHeader("nombre", valor)**
 - Para campos de tipo int realiza una conversión
 - e.g. Recargar página cada 60 segundos
 - `response.setIntHeader("Refresh", 60);`

● Específicos

- **setContentType("tipo_mime")**
 - Tipo de los contenidos devueltos. Formato MIME
 - e.g. `setContentType("image/png")`
- **setContentLength(int)**
 - Tamaño en bytes de contenido del paquete
 - e.g. `setContentLength( new File("logo.png").length() );`



Construir el cuerpo de la respuesta

- **Dos formas según el tipo de dato**
 - El objeto `HttpServletResponse` devuelve un objeto diferente para enviar el contenido dependiendo del tipo
 - No se pueden utilizar simultáneamente
- **Datos de texto:** (html, json, xml, texto plano)
 - `java.io.PrintWriter` [getWriter\(\)](#)
 - Algunos métodos de `PrintWriter`:
 - `print(cadena/char)`, `println(cadena)`, `printf("plantilla", val1, val2..)`, `write(..)`, `close()`
- **Datos binarios:** (e.g imágenes, ficheros, etc.)
 - `ServletOutputStream` [getOutputStream\(\)](#)
 - Algunos métodos de `ServletOutputStream`
 - `print(...)`, `write(..)`, `close()`

25

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo: Formulario y Resultado

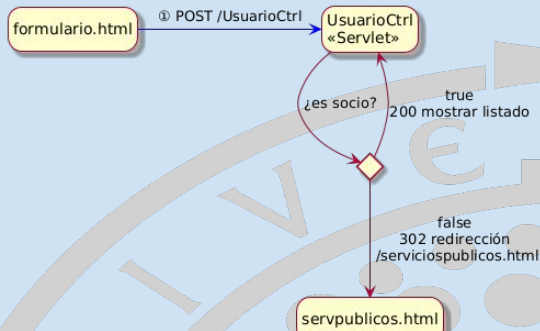
```
<form name="fDatosUsuario" action="UsuarioCtrl" method="POST">
  <fieldset>
    <legend>Datos del usuario</legend>
    <label>Nombre:<input type="text" name="nombre"></label>
    <label>Apellidos:<input type="text" name="apellidos"></label>
    <label>¿Es socio?
      <input type="checkbox" name="esSocio" value="Si"/>
    </label>
  </fieldset>
  <input type="submit" name="enviar" value="Enviar">
  <input type="reset" name="limpiar" value="Limpiar">
</form>
```

Alta de usuario

Datos del usuario

Nombre: Pedro Apellidos: Serrano Alba ¿Es socio? ☒

Enviar Limpiar



Redirigir a listado de servicios públicos

Bienvenido Pedro Serrano Alba

Listado de Servicios Socios

- Curso Padel Básico
- Curso Padel Avanzado

26

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo: Servlet

```
@WebServlet("/UsuarioCtrl")
public class UsuarioCtrl extends HttpServlet {
    private static List<String> servicios=new ArrayList<String>();

    @Override
    public void init() {
        servicios.add("Curso Padel Básico");
        servicios.add("Curso Padel Avanzado");
    }
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        try {
            if (request.getParameter("esSocio")!=null) {
                out.println("<!DOCTYPE html>"
                    + "<html><head><title>Servlet UsuarioCtrl</title></head><body>");
                out.printf("<h1>Bienvenido %s %s</h1>",
                    request.getParameter("nombre"),
                    request.getParameter("apellidos"));
                out.println("<h2>Listado de Servicios Socios</h2><ul>");
                for (String serv: servicios) {
                    out.printf("<li>%s</li>",serv);
                }
            } else {
                response.sendRedirect("servpublicos.html");
            }
            out.println("</html></body>");
        } finally {
            out.close();
        }
    }
}
```

27 }

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Java Server Pages

JSP

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



JSP

- **Jakarta Server Pages**

- Versión actual 3.1 JakartaEE 10

- **Definición**

- Un documento .jsp se puede entender como un **documento html que contiene código Java "embebido"**
 - Puede contener también otros tipos: xml, json, etc.
- El código Java que contiene permite **generar dinámicamente partes de dicho documento**
- **Facilita la creación de contenidos HTML** separando:
 - presentación: diseñadores gráficos
 - implementación: equipo de desarrollo
- Pueden ser editados por herramientas de edición html

29

Diferencias entre Servlet y JSP

- Un **Servlet** es adecuado para procesar la petición HTTP pero es complejo de usar a la hora de generar el contenido html
- Un documento **JSP** es una abstracción de más alto nivel que un Servlet, centrada en el contenido y no en el procesamiento
 - Servlet: código java con html embebido
 - JSP: html con código java embebido
- Se colocan en la raíz de contenidos web como otro fichero html, js, css, etc. cualquiera
- Se accede por su nombre de archivo
 - <http://www.ujaen.es/servicio/secretaria.jsp>

30

Ejemplo de JSP

Acceso:

`http://host/app/datosusuario.jsp?nombre=Pepe&apellidos=Lopez%20Garcia&esSocio=si`

```
<!DOCTYPE html>
<html lang="ES">
<head>
<title>Ejemplo</title>
<meta charset="UTF-8">
</head>
<body>
  <h1>Datos del usuario</h1>
  Nombre: <%= request.getParameter("nombre")%><br>
  Apellidos: <%= request.getParameter("apellidos")%><br>
  Es socio? <%= request.getParameter("esSocio")%><br>
  Hora actual: <%= new java.util.Date()%> <br>
  <footer>Conexión realizada desde <%=request.getRemoteAddr()%></footer>
</body>
</html>
```

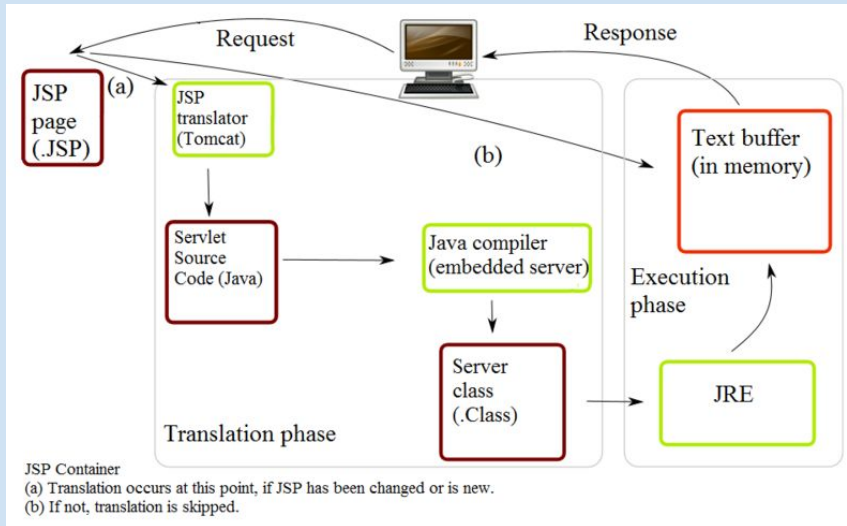
31

Arquitectura JSP

- Una página JSP se transforma en servlet la primera vez que se accede a ella:
 - Las etiquetas específicas se insertan en el código Java del servlet generado
 - El código html se inserta en instrucciones del estilo `out.println("contenido html")`
- Para el resto de peticiones se ejecuta directamente el **servlet generado**: eficiencia
- El servlet se vuelve a regenerar tras el reinicio del servidor o cuando se vuelve a editar la página .jsp original

32

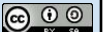
Ciclo de vida de un fichero .JSP



Fuente: Retrieved February 2019 from http://en.wikipedia.org/wiki/Java_Servlet

33

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

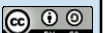


Elementos de una página JSP

- Una página JSP está formada por los siguientes elementos:
 - **Contenido estático**: normalmente HTML pero puede contener otros como XML, JSON, etc.
 - **Fragmentos de código java**, delimitado con unas marcas específicas `<% %>`
 - conocidos como *scriptlets*
 - Pueden ejecutarse en cada petición HTTP a la página
 - La salida que producen se envía junto al contenido estático
 - **Directivas**, aportan información al contenedor de servlets para su procesamiento. e.g. tipo MIME que devuelven
e.g. `<%@page contentType="text/html"%>`
 - **Etiquetas JSP**
 - Sintaxis parecida a etiquetas html
 - Usan un prefijo (espacio de nombres). e.g. jsp, c, fmt, etc.
 - Se ejecutan para cada petición e.g. `<jsp:include page="otrapagina.jsp"%>`
 - Pueden personalizarse, e.g. `<ficha-cliente datos="..."%>`

34

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Etiquetas para código Java

- Comentarios

- `<!-- comentario -->`

- Expresiones

- `<%= expresión_java %>`
- Se evalúan, convierten a String y se añaden a la salida del servlet. Equivale a `out.print(expresión)`. e.g. `<%= LocalDate.now() %>`

- Scriptlets

- `<% código java %>`
- código java que se coloca tal cual en el método de procesamiento de la petición del servlet

- Declaraciones:

- `<#! declaraciones %>`
- Insertadas en el servlet fuera de sus métodos. e.g. atributos del servlet, constantes e incluso métodos!

- **nota:** Si aparece `<%` en el texto html debe escaparse con `<\\%`

EL USO DE CÓDIGO JAVA EN
FICHEROS .JSP ESTÁ
DESACONSEJADO

35

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Variables Predefinidas

- Permiten comunicarse desde la página JSP con los elementos del Servlet asociado

- **request**

- parámetro `HttpServletRequest` del método `service`

- **response**

- parámetro `HttpServletResponse`

- **out**

- Writer del servlet

- **session**

- Objeto `HttpSession` para manejo de información de sesión

- **application**

- Objeto `ServletContext` obtenido por método `getServletContext()`

36

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo

<http://localhost:8080/dawsample/Usuario.jsp?nombre=Carlos&apellidos=Moreno&esSoci>

```
<%-- Ejemplo de atributo de servlet: compartido por todas las conexiones al servlet (hebras) --%>
<%! private int numAccesos=0;%> <%-- Define atributo en Servlet generado --%>
<!DOCTYPE html>
<html><head><title>Datos Usuario</title></head>
<body>
<% String[] servicios={"Curso de paddle","Alquiler de pistas","Piscina"};
String nombre=request.getParameter("nombre");
String apellidos=request.getParameter("apellidos");
if (nombre==null || nombre.trim().equals("")) nombre="Desconocido";
if (apellidos!=null) apellidos.trim();
boolean esSocio=(request.getParameter("esSocio")!=null?true:false);
%>
<h1>Bienvenido <%=nombre%> <%=apellidos%></h1>
<%if (esSocio) {%>
<h2>Listado de Servicios Socios</h2>
<ul>
<%for(String servicio: servicios) {%>
<li><%=servicio%></li>
<%}%>
</ul>
<%}%>
<p>Eres el visitante <%=++numAccesos%></p>
</body></html>
```

Bienvenido Carlos Moreno

Listado de Servicios Socios

- Curso de paddel
- Alquiler de pistas
- Piscina

Eres el visitante 10

37

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Directivas

- Proporcionan detalles de **configuración** para el servlet asociado
- **Directiva page** `<%@page atributo="valor"%>`
 - Determina características del servlet que se genera a partir de la página.
 - Algunos de los aspectos que permite controlar:
 - Clases que deben importarse
 - `<%@page import="pkg.class1,pkg.class2,..."%>`
 - Tipo MIME de los contenidos generados
 - `<%@page contentType="text/html"%>`
 - Si el servlet debe conservar la sesión del usuario
 - `<%@page session="true"%>` true, valor por defecto.
 - Qué página se mostrará si ocurre algún error inesperado (excepción)
 - `<%@page errorPage="/WEB-INF/erroresesperado.jsp"%>`
 - Páginas de tratamiento de errores se identifican con
 - `<%@page isErrorPage="true"%>`
 - disponen de un objeto excepción con info `<%= exception%>`

38

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Directivas (continuación)

● Directiva include `<%@include file="fichero"%>`

- Incluye (copia) el contenido de un fichero en la página durante la generación del servlet:

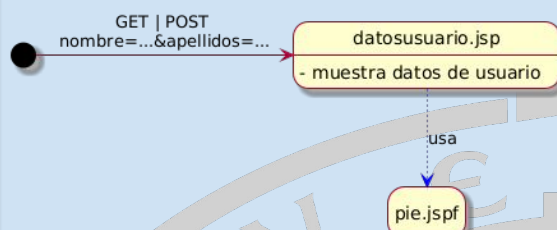
```
<%@include file="/WEB-INF/includes/pie.jspf"%>
```

- Es equivalente a la instrucción `#include` de C/C++
- La página incluida puede tener etiquetas JSP
- Extensión `.jspf` (fragmento jsp)
- **Inconveniente:** si se modifica el fichero incluido se debe volver a generar el servlet del fichero `.jsp` original (no se detecta automáticamente)
- **Alternativa:** mediante la etiqueta acción `<jsp:include page="url_relativa">`. Veremos su uso más adelante

Ejemplo de directivas

Fichero datosusuario.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html><head><title>Datos Usuario</title></head>
<body>
<h1>Datos del usuario</h1>
Nombre: <%= request.getParameter("nombre")%><br>
Apellidos: <%= request.getParameter("apellidos")%><br>
<%@include="/WEB-INF/jspf/pie.jspf"%>
</body></html>
```



Fichero pie.jspf

```
<%@page import=" java.util.Date"%>
<!-- Atributo de Servlet en JSP que incluya este fragmento-->
<%! private int numAccesos=0;%>
<footer>
<b>Club Squarepants</b>
Hora actual: <%= new java.util.Date()%> Número accesos: <%= ++numAccesos%>
</footer>
```


Conclusiones

- **Jakarta EE**, es un conjunto consensuado, por agentes importantes de la industria, de **especificaciones de módulos y APIs** para el desarrollo de aplicaciones empresariales en la plataforma JVM
- Un **servidor de aplicaciones JEE** pone a disposición de las aplicaciones los módulos definidos en su especificación.
- Un **contenedor de servlets** es una versión simplificada de un servidor de aplicaciones que da soporte un número reducido de módulos del estándar
- La especificación de **Servlets** define el componente básico para el tratamiento de peticiones HTTP en una aplicación web
- Las **páginas JSP** son un mecanismo para facilitar la generación de contenidos html dinámicos en una aplicación web

Bibliografía

- D. R. Heffelfinger, Servlet Development and Deployment, en *Java EE 8 Application Development*. Packt Publishing, 2017. [[Recurso electrónico](#)]
- Sierra, K, Basham, B. *Head First Servlets and JSP*. 2nd Edition. cap. 1, 4 y 7. O'Reilly Media, 2008. [[Recurso electrónico](#)]
- Groussard, T. (2011) *Java Enterprise Edition. Desarrollo de aplicaciones web con javaEE 6*. Capítulo 3 y 5. Ediciones ENI

Lecturas Complementarias

- *Jakarta EE Web profile: Servlets*, en Jakarta EE Tutorial, n.d. Eclipse Foundation. Retrieved from <https://jakarta.ee/learn/docs/jakartaee-tutorial/current>
- *Jakarta Servlet Specification 6.0*. Eclipse Foundation, 2022. Retrieved from <https://jakarta.ee/specifications/servlet>
- *Jakarta Server Pages 3.1*. Eclipse Foundation, 2022. Retrieved from <https://jakarta.ee/specifications/pages>